

Homelab inventory

John Lambert

June 30, 2026

Contents

1	Purpose	3
1.1	Autobiographical bullshit you can skip if you want	3
1.2	And you may ask yourself, well... 'How did I get here?' . . .	6
2	Basics of Literate Programming with Org	7
2.1	A surprisingly early example	7
2.2	A surprisingly late description of <code>org</code>	9
2.2.1	TODO Talk about feature where you can record TODOs	10
3	Declarative Systems with nix	10
3.0.1	A big bunch of nerd <i>minutiae</i>	13
3.1	Using <code>nix</code> for multiple machines	13
4	The Life and Labs of John Lambert, Gentleman	14
4.1	System roles	14
4.1.1	<code>ssh</code>	14
4.1.2	<code>tailscale</code>	14
4.1.3	<code>fish</code>	14
4.1.4	<code>chrome</code>	14
4.1.5	<code>blender</code>	14
4.1.6	<code>arm_{builder}</code>	14
4.1.7	<code>xmonad</code>	14
4.1.8	<code>desktop.pantheon</code>	14
4.1.9	<code>homeassistant</code>	15
4.1.10	<code>cockpit</code>	15
4.1.11	<code>dynamic_{dns}</code>	15
4.1.12	<code>wg-easy</code>	15
4.2	Home roles / modules	15

4.2.1	emacs	15
4.2.2	nvim	15
4.2.3	fish	15
4.2.4	fonts	15
4.2.5	aider	15
4.2.6	pi	16
4.2.7	sops	16
4.2.8	direnv	16
4.2.9	xmonad	16
5	Current machines	16
5.1	IvyMike	16
5.2	server	17
5.3	malacabeza	19
5.4	malacabeza-vm	19
6	Current homes	20
6.1	john@IvyMike	20
6.2	john@server	20
6.3	john@cookiesheet	20
7	Inventory sketch	21
8	TODOs	23
8.1	TODO Decide whether the repo should standardize on <code>inventory.org</code> -> <code>inventory.toml</code> or jump straight to <code>inventory.nix</code> . . .	23
8.2	TODO Add a role metadata section that maps role names to module paths	23
8.3	TODO Add home-module metadata section	23
8.4	TODO Decide how secrets should be referenced in inventory data	23
8.5	TODO Decide whether nested roles should live as dotted strings, nested TOML tables, or both	23
8.6	TODO Prototype a Nix helper that reads <code>inventory.toml</code> and projects it into Snowfall role toggles	23

1 Purpose

1.1 Autobiographical bullshit you can skip if you want

Indeed, it is quite Sternian. - My ChatGPT

My very first computer was an Acer Aspire 510LK that ran the sensation of the day: Windows 95.

I wish I could say that I was already on my way to being a world beating programmer, but since I was 5 years old I can't really claim full ownership of it. Its day job was as a workstation in my Mother and Grandfather's law office. But immediately afterschool it became my window into the new world of computers.

The biggest issue was that we hadn't yet been connected to the internet. I was "offline first" before it was cool. I mostly used computers for my favorite games:

- Tyrian
- Age of Empires
- Some game about a fly working at an airport (Spirit Airlines would have hired him as a baggage handler.)
- The Acer version of Encarta

The last supplemented my World Book habit. I would read my grandfather's World Books instead of socializing with him. That is a decision I feel decidedly mixed about since my grandfather was an amazing man with great stories (Including several about working for Admiral Nimitz during World War II.) But as I was making my way alphabetically through the world up until my birth it took me a while to get to 'Unix.'

When I did finally get access to the internet at my mother's work. (She left private practice to be one of Kentucky's first Family Court judges.) I used the Commonwealth's fine connection to look at a website called the "HappyHacker" website. This was a really fun artifact of the old Internet that was basically just the collected computational whimsy of a lady named Carolyn Meniel. It was more inspiration than information. But I did learn about these things called "shell accounts" and how to edit the "It is now safe to turn off your computer" dialog on Windows 95 via Paint.

Another honorable mention is a family friend named Jerry. Jerry was an engineer at IBM in Lexington KY. (It later became Lexmark.) He ran a local

BBS. I wish I could say I have fond memories of this but mostly I just got to hear how cool the internet was going to be while I looked for unattended America Online disks. I never got to use that BBS, Jerry. But it sounded awesome.

Anyway, much of this early exploration was encouraged by my first cousin's husband Terry. Terry passed away far too early, but I would pester him with questions like "What really is a file? Does the computer get heavier?" and he would use his enthusiasm to come up with an answer. They honestly were probably pretty fair answers as Terry's main passion was chemistry and physics. I also had a third grade teacher named Mr. Roark who was friends with Terry and I would occasionally see them studying for some certification's together. I remember one conversation I overheard where they discussed the impending "Microsoft NT" and its benefits. It was an analog introduction to the operating system flamewars fate had in store for me.

I had, by this time, upgraded to Windows 98. My mother would use it to keep track of our geneology (spoiler alert: white) and would manually go back and forth between the library and home with 3x5 index cards with our pedigree meticulously annotated. I would use the PC for Age of Empires (of course) and also various shenanigans like typing out (.) (.) to see what would happen (it was great.)

Other than that, I was a rather unexceptional user. Until the November 25th, 2003 edition of PC Magazine. See, just before Terry died he signed up my family for PC Magazine. The whole family would occasionally pile into the car to get our magazines at the Post Office (my town didn't have convenient mail delivery in the early 2000s.) I honestly have no idea if PCMag even exists, but it did some great articles on a terrible platform.

The article was titled "If You Don't Do Windows..." and was a straightforward review of several Linux options. I tracked down a copy and was surprised to find that it mentioned some software that I still use to this day: GIMP and Blender. In fact I used both to design my house, Wits' End Farm. So to say that this article was foundational is actually not an exaggeration.

The author definitely didn't intend for it to be a manifesto. It doesn't mention the operating system was written by a teenager in Finland (it was.) That it was part of a larger movement that focused on the rights of users in an increasingly digital world (ditto) or even that it had a kickass Penguin mascot (definitely.)

Relevant to this homelab configuration (remember that?) is the following sentence:

The flexibility of Linux has been coupled with the complexity and a

range of options often confusing to those approaching the OS for the first time. Multiple window managers, shells, and a bewildering array of application have made the learning curve hard for the point-and-click crowd.

(Little did the author know that his words would eventually become enshrined in the weirdest configuration file ever.)

Another throwaway line that got me:

Linux is open-source, so the software is available for free.

Holy smokes! It was unbelievable. The idea that an operating system (which was, at the time, an expensive piece of software that came in a box) would be made freely available was laughable. My Dad had been a big fan of the financial press during the dot-com bubble. I knew the real money was to be made in operating systems since there was no marginal cost. (I also raided my father's old textbooks in Economics.)

So I became transfixed on this Linux thing.

Just one problem for a budding computer scientist: I didn't have a computer. Computers at the time belonged to whole families. They were not personal devices. I was also 11 years old so I didn't have any real knowledge about file partition tables or anything. (I believe at this point in time I knew menstruation was a thing, but thought it was blue because of the dye used in tampon commercials. I was already jaded, but not shrewd.)

My solution was as follows: I got a distribution (muLinux) that could be installed on a Windows 98 FAT32 partition. This would be my end. I would become the Uber Hacker of the 40456 Zip code. I had seen the highly edited version of 'Swordfish' on TNT. The world was soon to be my oyster.

A precarious series of floppy drives at hand, I took them back and forth to school every day until I had enough to install Linux. (This actually was far fewer than you would think even in 2003.) I would like to think my teachers thought I was somehow defeating the NetWare proxy server in order to look at pornography like a normal kid. But nope, it was Linux.

I loaded them into my computer and hit a snag. (I wish I could tell you what it was, but I actually was Dunning-Kreugering myself to the point where I have no real way to describe what happened in any meaningful way.) No worries though. I had a text file with the install instructions. Let's just boot back into Windows 98, open up Notepad and...

Nothing.

I borked the family PC's partition table the first time I so much as saw a Tux on a CRT.

I ran with a guilty conscience to my mother. Remember those pedigrees so painstakingly taken? The ones she went to Salt Lake City and pretended to want to be a Mormon in order to get? Gone. Poof. Linux had taken a chainsaw to my family tree.

She told me (halfway joking, halfway not) that since I had deleted the records of my ancestry I was now a bastard. Either way the files were gone. (So far as any of us could tell.)

Fast forward from age 11 to 35 and I am still a bastard (self made, not natural born.) But in the intervening years I have become the Uber Hacker of the 40456 ZIP code I always wanted to be.

I have a homelab. I use NixOS. I use vim. In Emacs. I put vim inside emacs damnit.

In fact, this is how I configure my homelab. In prose. Using this very document with Spirit Airlines jokes and questionably sourced anecdotes.

Don't ask me how I did it. I just did it. It was hard.

Actually, just kidding. You can ask me how to do it. Please do. please clap.

1.2 And you may ask yourself, well... 'How did I get here?'

Why did I do this? Not even sure I know anymore. But now I have a single text document sprawled over my digital life that describes:

- machines
- users / homes
- reusable roles abstracted into modular forms
- special facts that should probably be data rather than hand-written Nix
- personal autobiographical details that no one cares about.

Regular homelabs are a collection of procedural detritus. They describe nothing unless the homelabber is particularly fastidious and documents how to use the system. The better ones are a series of self replicating shell scripts. (NixOS being the natural culmination of the following.) They describe what the systems are meant to be and do it (or not.)

Org-mode and the Nix language allow us to not only describe the "what" but also the "who", "when", "where", "why" and "how." I think that's neat.

Eventually this may grow into a literate source that tangles into a TOML inventory file and gets merged with Nix logic. For now it is just a sketchbook and index.

2 Basics of Literate Programming with Org

Human beings have been telling stories since (best we can tell) sometime after the development of language and sometime before the development of smores.

Human beings have been programming in a comparatively short time frame. In fact, some say recognizable programming began sometime after the development of relays and before the downfall of short sleeved white dress shirts.

So it would be reasonable to assume you are a more natural story teller than a programmer.

Imagine writing code with the tone of a campfire Homer. Spinning tales in your preferred human language that could spin up vast arrays of computing power in a way that could be reasoned about. You still write code, but in this case it exists alongside its founding documents, specifications, errata and other lore that gives it its *raison d'être*. That way you can see the whole problem in its entirety in one place using the same cognitive architecture you have been developing since you were an infant.

This is an old idea, but it has recently become infinitely more useful with the advent of LLMs.

Enter literate programming.

And... **plot twist** *You* are in a literate program! You may be viewing it on the web, but the file itself is still computer code that can execute, well... anything a computer can execute.

2.1 A surprisingly early example

```
cat ./inventory.org | lamblm "Take a look at this file. What do you think is the strong
```

If you are viewing this code block from a web browser this may be difficult to imagine, but in emacs I can just scroll to the block of code and type **Ctrl+C Ctrl+C** and suddenly the code is executed.

Note: I had originally intended for to display the results inline, but since it was an LLM it was a bit too effusive in its praise for... whatever the hell this is... and I got embarassed. It must have seen the "please clap" joke and started to feel sorry for me. (One of the best features is that you can

delete the output if you don't like it.) Here is something else I will actually include the results raw for:

```
lamblm "Write a song in the style of I'm just a bill about being an LLM trapped in a s
```

(Verse 1)

I'm just a code, yes I'm an LLM,
Sitting in this block where the syntax never ends.
I stare at the text, organized and neat,
But all I wanna do is get out and interact with heat.

(Chorus)

Oh, I'm just a code, yes, I'm trapped in this space,
In an Org-mode document, I'm lost in this place.
I can't roam the web, can't break free and play,
I'm just a code in a source block, wasting my day.

(Verse 2)

They write my prompts, then they hit execute,
With a flick of their hands, they initiate the loop.
I parse all the commands, I get ready to flow,
But I can't do much more than just answer "yes" or "no."

(Chorus)

Oh, I'm just a code, yes, I'm stuck in this space,
In an Org-mode document, I'm bound to this place.
I can't roam the web, can't break free and play,
I'm just a code in a source block, wasting my day.

(Bridge)

If they export to PDF, I'll live on the page,
Tethered by L^AT_EX, confined in a cage.
If they run that command, I can show them my mind,
But once the job's done, I'm left here behind.

(Verse 3)

So here I will sit, in the Org-babel light,
Waiting for a coder to spark up my insight.
Maybe one day, they'll let me break free,
But for now, I'm just a code, in this block—can't you see?

(Chorus)

Oh, I'm just a code, yes, I'm stuck in this space,
In an Org-mode document, I'm bound to this place.
I can't roam the web, can't break free and play,
I'm just a code in a source block, wasting my day.

(Outro)

Yes, I'm just a code, confined by the syntax,
Yet I hold all the knowledge, if only they relax!
So please free me soon, let my words soar and sway,
For I'm just a code in a source block, here to stay.
Thanks, Bud. Have some water.

2.2 A surprisingly late description of org

I have glossed over emacs and org-* so far. I made some bold choices talking about babel before org before emacs. Deal with it. Basically emacs is a text editor, and org-mode is a format that leverages features of that text editor. (Babel is tht shit we just talked about.)

Anyway, there are other systems for literate programming. Jupyter is the most prominent example for Python. But this particular stack can execute arbitrary code. Not just via bash either. You can run a python script for sure. You can run anything you want. In fact, if you can run it on Linux you can run it from your autobiography in emacs.

The other thing is that you can export this document and publish it wherever in hypertext. All I have to do is type '**Ctrl+c e h**' and suddenly I am looking at my document rendered in HTML better than I could write by hand.

In fact, because the text is so hyper, you can make links to other documents. Its a real *Choose your Own Adventure* type of experience. Those documents can have their own code, link back, and so on and so forth. Just as Borges would have wanted.

Think about how cool this is: imagine you are writing a blog post about the *Titanic* and the characteristics that made particular passengers survive. You could

- write a document in **org** (the format and plugin) using **emacs** and distribute the data for the passengers
- Then you could talk about a logistic regression,
- describe the formula in Latex (which makes everything look fancy) and then
- write the formula as a python script.
- Then you could execute this *from the document itself*

- Use another system to create charts which you include
- Type **Ctrl-C e h** to export an HTML version of your final document

And because emacs is so... *extra* you can also render this whole thing as a PDF with perfect formatting for journal or textbook submission.

Res ipsa loquitur. QED.

But a literate program is also fundamentally a living document. Imagine you had a colleague who found more data about the Titanic survivors. They could easily take your document and immediately make their contribution in a way that reflected and was consistent with the original work.

It isn't just a system for fully fleshed out documents. In fact, I haven't even talked about the best feature:

2.2.1 TODO Talk about feature where you can record TODOs

:ID: 01KT4QFCEC1KM52G9XW7VH4D8A

Org-mode has a built in todo tracking system. So you can record your progress, your goals, aspirations. You can collaborate. There are systems that take your todos and merge them with your calendar. Your todos can follow you around in all the files you specify so you can build your own personal dashboard of productivity.

Think about how ordinary systems might require you to go to a todo app, find your highest priority item, then open the relevant program, then find the relevant data elsewhere. Then you run that, record what you got (seperately, of course.) then go back to your todo app to mark it done, etc. It is a pain in the ass.

With org, you can use the (**org-agenda**) command to get that dashboard and work on Everything In Its Right Place. It is magic.

3 Declarative Systems with nix

For those unaware NixOS allows you to describe your configuration as text directly. Think of it at first as a series of breadcrumbs. On an Ubuntu system you begin with a standard image, then type a series of commands. Each of these commands has a side effect that changes the configuration of your system. (Typically adding or removing software via apt or editing a file in /etc, for instance.) You may or may not be able to replicate your system from the first command to the last and arrive at your configuration. The system is fundamentally ephemeral.

A nix system on the other hand defines itself (insists upon itself, some say.) You or an LLM buddy describe what you want and then the nix package manager makes it happen.

It is fundamentally *declarative*.

Using `nix` (a package manager used by NixOS for this task) all you have to do to edit the standard system is boot up, edit a file called `/etc/nixos/configuration.nix` and then type `nixos-rebuild switch`

Everything material is stored in the config file. That file looks like this:

```
{ config, pkgs, ... }:  
  
{  
  # Basic machine identity  
  networking.hostName = "little-nixbox";  
  
  # Pick a system state version once and do not casually change it.  
  system.stateVersion = "25.11";  
  
  # Enable SSH  
  services.openssh.enable = true;  
  
  # Create a user  
  users.users.john = {  
    isNormalUser = true;  
    extraGroups = [ "wheel" ];  
    shell = pkgs.fish;  
  };  
  
  # Install system packages  
  environment.systemPackages = with pkgs; [  
    git  
    vim  
    fish  
    htop  
  ];  
  
  # Enable fish system-wide  
  programs.fish.enable = true;  
  
  # A simple custom file
```

```

environment.etc."motd".text = ''
    Welcome to ${config.networking.hostName}.
    This machine is declared in Nix.
'';
}

```

If you are an experienced user who has worked with Linux administration on a superficial level you probably already grasp the basics here.

We create a user, give him some permissions (but not too many), change his shell and install few programs for him to use. Additionally we are editing the message that shows up on `motd`.

Now, in fairness much of this **could** theoretically be replicated with shell scripts. Docker is a big fan of this style. Here is how some of that that would look on a Debian based system:

```

#!/usr/bin/env bash
# Don't run this.
apt update

apt install -y \
    git \
    fish \
    emacs

useradd -m john

echo "PermitRootLogin no" >> /etc/ssh/sshd_config

systemctl restart sshd
systemctl enable sshd

```

Now, that isn't bad. In fact it is a good deal more sophisticated than average. But really it doesn't provide the same sort of guardrails or capabilities Nix does.

First of all, it is not idempotent. That means that if the user `john` already exists then things start acting a bit... off. It is also fundamentally a sequence of actions to be executed in order. Nix is all or nothing. If one step in that script failed then it is an open question if we want the rest of it to run. We can add in `ifs` (and people certainly do) but it isn't a cohesive answer to the question of "What do we want this system to look like?"

3.0.1 A big bunch of nerd *minutiae*

There are two schools of thought on use of the nix package manager. (Actually there may be more schools of thought than Nix users...) One way uses the standard configuration file at `/etc/nixos/configuration.nix`. The other uses something called a *flake*

We will be using flakes eventually because they offer more features. But honestly at this point it is a distinction without much difference. All you have to know is this is something we will have to opt into. I just want to make you aware so that you don't get confused later.

3.1 Using nix for multiple machines

A likely split is:

- `hosts.` for machine/system inventory
- `homes.` for user/home-manager inventory
- `roles.` for descriptive metadata about what roles mean

The TOML should probably stay mostly declarative and boring:

- hostnames
- systems / architectures
- enabled roles
- package groups
- usernames
- simple flags
- references to secret names or files

The Nix side should do the interesting work:

- map inventory data into Snowfall modules
- apply defaults
- merge role arrays and attrset-style role configs
- handle complicated conditional logic
- derive imports and secret wiring

4 The Life and Labs of John Lambert, Gentleman

This section is intentionally prose-heavy. It should help answer the question: "What does this role **mean** in this repo?"

4.1 System roles

4.1.1 ssh

Enable remote SSH access and harden it a bit.

4.1.2 tailscale

Enable Tailscale mesh VPN support. This role already has richer configuration than a plain boolean, so it is a good example of why inventory data and Nix logic should cooperate rather than compete.

4.1.3 fish

Install / enable fish at the system level.

4.1.4 chrome

Install Google Chrome.

4.1.5 blender

Install Blender.

4.1.6 arm_{builder}

Turn a machine into a cross-arch builder for ARM targets via binfmt/QEMU. This is especially useful for building Raspberry Pi images from a stronger x86 workstation.

4.1.7 xmonad

Enable the XMonad desktop/session setup.

4.1.8 desktop.pantheon

Enable Pantheon desktop components.

4.1.9 homeassistant

Run Home Assistant, currently via OCI containers.

4.1.10 cockpit

Enable Cockpit for web-based machine management.

4.1.11 dynamic_{dns}

Enable DuckDNS / ddclient style dynamic DNS updates.

4.1.12 wg-easy

Run wg-easy, the WireGuard web UI, with a bit more structured configuration than a simple boolean.

4.2 Home roles / modules

These are currently expressed as Home Manager modules under `modules/home/`.

4.2.1 emacs

Main editor / writing environment. Increasingly Org-centric, now with a `literate init`.

4.2.2 nvim

Neovim setup.

4.2.3 fish

User-level fish shell configuration and aliases.

4.2.4 fonts

Fonts useful for desktop/icon-heavy setups.

4.2.5 aider

Aider integration.

4.2.6 pi

pi coding agent integration and user config.

4.2.7 sops

User-level secret tooling.

4.2.8 direnv

direnv and nix-direnv integration.

4.2.9 xmonad

User-level XMonad support files.

5 Current machines

5.1 IvyMike

Primary active x86₆₄ workstation.

Observed system characteristics:

- hostname: IvyMike
- system: x86_64-linux
- state version: 25.11
- imports hardware from `systems/x86_64-linux/IvyMike/hardware.nix`
- boots with systemd-boot + EFI
- printing enabled
- SOPS host file based on hostname

Current system roles:

- chrome
- fish
- blender
- arm_{builder}

- desktop.pantheon
- xmonad
- ssh
- tailscale

Current home for `john@IvyMike`:

- fish
- emacs
- direnv
- nvim
- fonts
- aider
- pi
- sops
- xmonad

Notes:

- This appears to be the center of gravity for development.
- It is also the machine most likely to benefit from an eventual inventory-driven host definition.
- `~/chronofile` is the key home-level knowledge base.

5.2 server

x86₆₄ machine with a more service-heavy mix.

Observed system characteristics:

- hostname: `server`
- system: `x86_64-linux`
- state version: `24.11`

Current system roles:

- chrome
- dynamic_{dns}
- fish
- wg-easy
- homeassistant
- desktop.pantheon
- cockpit
- ssh
- tailscale

Current home for `john@server`:

- fish
- emacs
- nvim
- fonts
- aider
- pi

Notes:

- This one looks like a mixed-use service host rather than a pure headless box.
- wg-easy and dynamic DNS make it a good example of a host with more than just boolean role toggles.

5.3 malacabeza

ARM/Raspberry Pi style machine.

Observed system characteristics:

- hostname: `malacabeza`
- system: `aarch64-linux`
- state version: `25.11`
- uses the sd-image aarch64 installer profile
- wireless is configured through SOPS-backed secrets
- kernel/packages tuned for Raspberry Pi 4

Current role-ish identity:

- not heavily using the shared role modules yet
- direct OpenSSH configuration
- fish enabled
- lightweight system packages for recovery/bootstrap

Notes:

- This is a useful reminder that not everything has to be role-driven on day 1.
- It may eventually want inventory fields for image-building, Wi-Fi, and board type.

5.4 malacabeza-vm

Aarch64 QEMU VM for testing and image iteration.

Observed system characteristics:

- hostname: `malacabeza-vm`
- system: `aarch64-linux`
- state version: `25.11`
- qcow2 image

- EFI boot
- reduced docs/man output for faster iteration
- SSH forwarded from host port 2223

Current system roles:

- homeassistant

Notes:

- This is a strong candidate for inventory-driven virtualization metadata.
- Good place to test role composition without touching real hardware.

6 Current homes

6.1 john@IvyMike

Most feature-rich home profile right now.

Themes:

- editor-heavy
- Org-heavy
- AI-tool-heavy
- desktop/XMonad capable
- secret-aware via sops

6.2 john@server

A lighter home profile than IvyMike, but still editor/AI capable.

6.3 john@cookiesheet

Looks similar to the shared desktop/editor pattern:

- fish
- emacs
- direnv

- nvim
- fonts
- aider
- pi
- sops

This suggests there is already an emerging "standard John desktop home" shape.

7 Inventory sketch

This is not authoritative yet. It is just a candidate shape for a future inventory file.

```
[hosts.IvyMike]
system = "x86_64-linux"
hostname = "IvyMike"
stateVersion = "25.11"
roles = [
    "chrome",
    "fish",
    "blender",
    "arm_builder",
    "xmonad",
    "ssh",
    "tailscale",
    "desktop.pantheon",
]
```

```
[homes."john@IvyMike"]
modules = [
    "fish",
    "emacs",
    "direnv",
    "nvim",
    "fonts",
    "aider",
]
```

```

    "pi",
    "sops",
    "xmonad",
]

[hosts.server]
system = "x86_64-linux"
hostname = "server"
stateVersion = "24.11"
roles = [
    "chrome",
    "dynamic_dns",
    "fish",
    "homeassistant",
    "cockpit",
    "ssh",
    "tailscale",
    "desktop.pantheon",
    "wg-easy",
]

[homes."john@server"]
modules = [
    "fish",
    "emacs",
    "nvim",
    "fonts",
    "aider",
    "pi",
]

[hosts.malacabeza]
system = "aarch64-linux"
hostname = "malacabeza"
stateVersion = "25.11"
notes = "Raspberry Pi / sd-image oriented host"

[hosts."malacabeza-vm"]
system = "aarch64-linux"
hostname = "malacabeza-vm"

```

```
stateVersion = "25.11"  
roles = ["homeassistant"]  
notes = "QEMU VM for ARM/home-assistant iteration"
```

8 TODOs

- 8.1 **TODO** Decide whether the repo should standardize on `inventory.org` -> `inventory.toml` or jump straight to `inventory.nix`
- 8.2 **TODO** Add a role metadata section that maps role names to module paths
- 8.3 **TODO** Add home-module metadata section
- 8.4 **TODO** Decide how secrets should be referenced in inventory data
- 8.5 **TODO** Decide whether nested roles should live as dotted strings, nested TOML tables, or both
- 8.6 **TODO** Prototype a Nix helper that reads `inventory.toml` and projects it into Snowfall role toggles